DX Framework учебник по математике и исходному коду

Учебное издание 9.0

Математика от ручного расчёта до DirectXMath

Построчное чтение текущих ECS боевых и тактических систем

Справочник алгоритмов и учебный маршрут на четыре недели

Собрано по текущему снимку репозитория

1 ноября 2026 года

Как пользоваться книгой

Эта редакция не повторяет старый справочник. Она построена вокруг четырёх действий: посчитать вручную, найти то же действие в текущем коде, изменить маленький участок самостоятельно и доказать результат отладкой.

Не читай книгу подряд как роман. После каждой темы выполняй указанное мини задание. Если не можешь предсказать результат до запуска, вернись к ручному примеру.

Построчные таблицы используют реальный код проекта. Колонка с поломкой нужна не для страха, а для понимания контракта строки. Именно контракт подсказывает, куда безопасно добавлять новую механику.

Карта книги

Раздел	Что ты научишься делать
Математика движка с нуля	Векторы матрицы лучи пересечения время и упражнения
Построчный разбор систем DX Framework	ECS Transform тактика укрытия состояние стрельба и урон
Справочник полезных алгоритмов	Готовые схемы выбора движения навигации и отладки
Учебный маршрут по DX Framework и NPC	Интенсив на день и практика на четыре недели

Условие честной проверки

Сначала записывай ожидаемое поведение, затем запускай сцену. Если результат отличается, ищи первую строку цепочки данных, где фактическое значение перестало совпадать с ожидаемым.

Математика движка с нуля

Эта глава не является списком формул. Каждая тема проходит один и тот же путь: смысл, ручной расчёт, запись на C++, применение в DX Framework и проверка результата. Если ручной расчёт не совпал с кодом, не переходи дальше.

1 Числа единицы измерения и погрешность

`float` хранит приближённое число. Поэтому координаты, длины и таймеры почти никогда не сравнивают с нулём через `==`.

```
constexpr float epsilon = 0.0001f;
const bool almostZero = std::fabs(value) <= epsilon;
```

Всегда подписывай единицы мысленно или в имени:

Величина	Единица	Пример
позиция	метры	`position = { 4, 1, -2 }`
скорость	метры в секунду	`runSpeed = 5.5f`
время кадра	секунды	`deltaTime = 0.016f`
угол DirectXMath	радианы	`XMConvertToRadians(45.0f)`
здоровье	условные единицы	`health = 100.0f`

Если скорость равна 5 м/с, а кадр длится 0.02 с, перемещение за кадр равно:

$$\text{deltaPosition} = \text{speed} \cdot \text{deltaTime} = 5 \cdot 0.02 = 0.1$$

Ошибка новичка: прибавлять `speed` каждый кадр. Тогда результат зависит от FPS.

2 Точка и вектор

Точка отвечает на вопрос «где». Вектор отвечает на вопрос «куда и насколько».

Пусть NPC находится в точке A = (2, 1, 3), а цель в B = (8, 1, 7). Направление от NPC к цели получается вычитанием начала из конца:

$$\mathrm{d} = \mathrm{B} - \mathrm{A} = (8-2, 1-1, 7-3) = (6, 0, 4)$$

```
XMFLOAT3 direction{
    target.x - npc.x,
    target.y - npc.y,
    target.z - npc.z
};
```

Перепутанный порядок даст вектор от цели к NPC. Это не «почти тот же результат»: знак определяет направление.

Сложение векторов

Складываем соответствующие координаты:

$$(a_x, a_y, a_z) + (b_x, b_y, b_z) = (a_x + b_x, a_y + b_y, a_z + b_z)$$

Пример: движение вперёд (0, 0, 3) и вправо (2, 0, 0) даёт (2, 0, 3).

Умножение на число

Каждая координата умножается на одно число:

$$2 \cdot (3,0,-1) = (6,0,-2)$$

Так из единичного направления получают скорость, а из скорости — смещение за кадр.

3 Длина нормализация и расстояние

Длина вектора вычисляется по теореме Пифагора:

$$|v| = \sqrt{(x^2 + y^2 + z^2)}$$

Для d = (6, 0, 4):

$$|d| = \sqrt{(6^2 + 0^2 + 4^2)} = \sqrt{(52)} \approx 7.211$$

Нормализация оставляет направление, но делает длину равной единице:

$$\text{unit d} = (d)/(|d|) \approx (0.832,0,0.555)$$

```
XMVECTOR raw = XMLoadFloat3(&direction);
const float length = XMVectorGetX(XMVector3Length(raw));

if (length > 0.0001f)
{
    XMVECTOR unit = XMVector3Normalize(raw);
}
```

Нулевой вектор нормализовать нельзя: деление на ноль породит `NaN`. Поэтому сначала проверяется длина.

Расстояние без квадратного корня

Если нужно только сравнить расстояние с радиусом, корень не нужен:

$$\text{distanceSquared} = dx^2 + dy^2 + dz^2$$

```
const float distanceSquared = dx * dx + dy * dy + dz * dz;
if (distanceSquared < attackRange * attackRange)
{
    // Цель внутри дальности.
}
```

Это дешевле и хорошо подходит для массового поиска кандидатов. Настоящую длину вычисляй только для финального кандидата или интерфейса.

4 Скалярное произведение и поле зрения

Скалярное произведение двух нормализованных векторов показывает, насколько одинаково они направлены:

$$a \cdot b = a_xb_x + a_yb_y + a_zb_z$$

Значение	Геометрический смысл
1	направления совпадают
0	угол 90 градусов
-1	направления противоположны

Связь с углом:

$$a \cdot b = |a||b|\cos(\theta)$$

Если оба вектора единичные, остаётся `cos(theta)`. Для поля зрения шириной 90 градусов половинный угол равен 45 градусов:

$$\text{threshold} = \cos(45°) \approx 0.707$$

```
const XMVECTOR forward = XMVector3Normalize(forwardVector);
const XMVECTOR toTarget = XMVector3Normalize(targetVector);
const float dot = XMVectorGetX(XMVector3Dot(forward, toTarget));
const float threshold = std::cos(XMConvertToRadians(45.0f));
const bool insideFov = dot >= threshold;
```

Важно: поле зрения ещё не доказывает видимость. После него нужен raycast, иначе NPC увидит цель сквозь стену.

5 Векторное произведение

Векторное произведение создаёт вектор, перпендикулярный двум исходным:

$$a \times b = (a_yb_z - a_zb_y, a_zb_x - a_xb_z, a_xb_y - a_yb_x)$$

В движке оно полезно, чтобы получить правое направление относительно `forward` и мирового `up`:

```
XMVECTOR right = XMVector3Normalize(XMVector3Cross(up, forward));
```

Порядок важен: `Cross(a, b)` и `Cross(b, a)` смотрят в противоположные стороны. Если правая точка выглядывания оказалась слева, первым делом проверь порядок аргументов и систему координат.

6 Что такое матрица

Матрица — прямоугольная таблица чисел. В 3D часто используется матрица 4 на 4: она помещает поворот, масштаб и перенос в одну операцию.

$$M = [\ m_(11), m_(12), m_(13), m_(14);\ m_(21), m_(22), m_(23), m_(24);\ m_(31), m_(32), m_(33), m_(34);\ m_(41), m_(42), m_(43), m_(44)\ ]$$

Индекс `m23` означает строку 2, столбец 3. Прежде чем программировать матрицы, научись уверенно находить ячейку по двум индексам.

Единичная матрица

Единичная матрица ничего не меняет:

$$I = [\ 1, 0, 0, 0;\ 0, 1, 0, 0;\ 0, 0, 1, 0;\ 0, 0, 0, 1\ ]$$

Она играет ту же роль, что число 1 при обычном умножении.

7 Как складывать матрицы

Матрицы можно складывать, только если у них одинаковое число строк и столбцов. Складывается каждая пара соответствующих ячеек.

$$A = [\ 1, 2;\ 3, 4\ ], B = [\ 5, 6;\ 7, 8\ ]$$

$$A + B = [1 + 5, 2 + 6;\ 3 + 7, 4 + 8] = [6, 8;\ 10, 12]$$

Псевдокод показывает главный принцип:

```
for (int row = 0; row < rowCount; ++row)
    for (int column = 0; column < columnCount; ++column)
        result[row][column] = a[row][column] + b[row][column];
```

Сложение матриц редко используется для построения `world`-матрицы. Трансформации обычно соединяются умножением. Но сложение важно знать для накопления поправок, смешивания данных и понимания линейной алгебры.

Умножение матрицы на число

Число умножается на каждую ячейку:

$$2A = [2, 4;\ 6, 8]$$

8 Как умножать матрицы

Чтобы вычислить одну ячейку результата, берём строку левой матрицы и столбец правой, перемножаем пары и складываем.

$$C_(row, column) = \Sigma_k\ A_(row, k) B_(k, column)$$

Рассмотрим полный расчёт:

$$A = [1, 2;\ 3, 4], B = [5, 6;\ 7, 8]$$

$$C_(11) = 1 \cdot 5 + 2 \cdot 7 = 19$$

$$C_(12) = 1 \cdot 6 + 2 \cdot 8 = 22$$

$$C_(21) = 3 \cdot 5 + 4 \cdot 7 = 43$$

$$C_(22) = 3 \cdot 6 + 4 \cdot 8 = 50$$

$$AB = [19, 22;\ 43, 50]$$

Матрицы не перестановочны: обычно `A * B != B * A`. Поэтому порядок `scale * rotation * translation` является частью поведения, а не косметикой.

```
for (int row = 0; row < 2; ++row)
{
    for (int column = 0; column < 2; ++column)
    {
        result[row][column] = 0.0f;
        for (int k = 0; k < 2; ++k)
            result[row][column] += a[row][k] * b[k][column];
    }
}
```

9 Масштаб поворот и перенос

В `TransformComponent::GetWorldMatrix` текущего проекта операции создаются отдельно:

```
XMMATRIX scaleMatrix = XMMatrixScaling(scale.x, scale.y, scale.z);
XMMATRIX rotX = XMMatrixRotationX(rotation.x);
XMMATRIX rotY = XMMatrixRotationY(rotation.y);
XMMATRIX rotZ = XMMatrixRotationZ(rotation.z);
XMMATRIX rotationMatrix = rotX * rotY * rotZ;
XMMATRIX translationMatrix = XMMatrixTranslation(position.x, position.y, position.z);
return scaleMatrix * rotationMatrix * translationMatrix;
```

Читаем не как «набор непонятных матриц», а как конвейер:

1. локальная вершина получает масштаб;
2. результат поворачивается по X, затем Y, затем Z в принятой проектом конвенции;
3. результат переносится в позицию сущности;
4. получается мировая координата.

Если сначала перенести объект, а затем повернуть, перенос тоже может повернуться вокруг начала координат. Поэтому объект начнёт двигаться по дуге вместо поворота на месте.

Координата и направление преобразуются по разному

`XMVector3TransformCoord` применяет перенос. Для точки дула это правильно. Направление не должно получать перенос; для него используют `XMVector3TransformNormal` или только поворотную часть.

```
XMVECTOR worldPoint = XMVector3TransformCoord(localPoint, worldMatrix);
XMVECTOR worldDirection = XMVector3TransformNormal(localDirection, worldMatrix);
```

10 Обратная матрица

Обратная матрица отменяет преобразование:

$$M^{(-1)}M = I$$

Если `world` переводит локальную точку в мир, `inverseWorld` возвращает мировую точку в локальное пространство.

```
XMMATRIX world = transform.GetWorldMatrix();
XMMATRIX inverseWorld = XMMatrixInverse(nullptr, world);
XMVECTOR localPoint = XMVector3TransformCoord(worldPoint, inverseWorld);
```

Практическое применение: определить, где попадание находится относительно отсека или корпуса, а не относительно всей сцены.

Не всякая матрица обратима. Нулевой масштаб по оси делает преобразование вырожденным. Поэтому масштаб `(0, 1, 1)` опасен для кода, которому нужна обратная матрица.

11 Пространства координат

Одна и та же точка проходит несколько пространств:

Пространство	Вопрос
локальное	где точка относительно модели
мировое	где точка на уровне
вид камеры	где точка относительно камеры
clip	как точка прошла проекцию
NDC	где точка в нормализованном экране
экранное	какой это пиксель

Типовой конвейер:

$$p_(clip) = p_(local) \cdot World \cdot View \cdot Projection$$

Если модель видна, но raycast идёт не туда, часто смешаны локальная и мировая координаты. Всегда спрашивай: «в каком пространстве находится эта переменная?»

12 Что такое луч

Луч задаётся началом O и нормализованным направлением D:

$$P(t) = O + tD, t \geq 0$$

`t` — расстояние вдоль луча, если D единичный. Для конечного отрезка вводится `0 <= t <= maxDistance`.

Пример: O = (1, 2, 3), D = (0, 0, 1). При t = 5 получаем P = (1, 2, 8).

```
XMFLOAT3 origin = muzzleWorldPosition;
XMFLOAT3 rawDirection{
    aimPoint.x - origin.x,
    aimPoint.y - origin.y,
    aimPoint.z - origin.z
};

XMVECTOR direction = XMVector3Normalize(XMLoadFloat3(&rawDirection));
```

Создание луча всегда состоит минимум из трёх вопросов:

1. Откуда начинается луч?
2. Куда он направлен?
3. Какова максимальная длина и кого он должен игнорировать?

Для винтовки начало должно быть у дула, а не у центра NPC или камеры. Иначе визуальный ствол и физический выстрел разойдутся.

13 Луч из курсора камеры

Для выбора объекта мышью нужно построить две мировые точки: на ближней и дальней плоскости камеры.

1. Перевести пиксель мыши в NDC.
2. Создать точки near и far в clip-пространстве.
3. Умножить их на обратную `ViewProjection`.
4. Разделить на `w`.
5. Началом сделать near, направлением — `normalize(far - near)`.

$$x_(ndc) = 2(x_(screen)/width) - 1$$

$$y_(ndc) = 1 - 2(y_(screen)/height)$$

Y переворачивается, потому что экранные координаты обычно растут вниз, а NDC — вверх.

14 Пересечение луча с плоскостью

Плоскость задаётся точкой Q и нормалью N. Подставляем луч в уравнение плоскости:

$$t = ((Q - O) \cdot N)/(D \cdot N)$$

Алгоритм:

1. вычислить denominator = dot(D, N);
2. если модуль denominator почти нулевой, луч параллелен плоскости;
3. вычислить t;
4. если t < 0, пересечение позади начала луча;
5. вернуть O + tD.

```
const float denominator = Dot(rayDirection, planeNormal);
if (std::fabs(denominator) <= 0.0001f)
    return false;

const float t = Dot(planePoint - rayOrigin, planeNormal) / denominator;
if (t < 0.0f)
    return false;

hitPoint = rayOrigin + rayDirection * t;
```

Применение: поставить маркер на полу, найти точку приказа, пересечь прицел с условной тактической плоскостью.

15 Пересечение луча со сферой

Сфера задаётся центром C и радиусом r. Ищем точку луча, расстояние которой до C равно r:

$$|O + tD - C|^2 = r^2$$

После раскрытия получается квадратное уравнение. Удобная реализация для нормализованного D:

```
const Vec3 oc = origin - center;
const float b = Dot(oc, direction);
const float c = Dot(oc, oc) - radius * radius;
const float discriminant = b * b - c;

if (discriminant < 0.0f)
    return false;

const float root = std::sqrt(discriminant);
float t = -b - root;
if (t < 0.0f)
    t = -b + root;
return t >= 0.0f;
```

Отрицательный дискриминант означает промах. Два корня — вход и выход из сферы. Обычно нужен ближайший неотрицательный.

16 Пересечение луча с AABB

AABB — коробка, выровненная по мировым осям. Slab-алгоритм рассматривает диапазон допустимых `t` для каждой оси и пересекает диапазоны.

```
float tMin = 0.0f;
float tMax = maxDistance;

for (int axis = 0; axis < 3; ++axis)
{
    if (std::fabs(direction[axis]) < epsilon)
    {
        if (origin[axis] < boxMin[axis] || origin[axis] > boxMax[axis])
            return false;
        continue;
    }

    float t1 = (boxMin[axis] - origin[axis]) / direction[axis];
    float t2 = (boxMax[axis] - origin[axis]) / direction[axis];
    if (t1 > t2)
        std::swap(t1, t2);

    tMin = (std::max)(tMin, t1);
    tMax = (std::min)(tMax, t2);
    if (tMin > tMax)
        return false;
}
```

Ключевая мысль: луч должен одновременно находиться внутри допустимого интервала по X, Y и Z.

17 Raycast в физическом мире

Математический луч только описывает линию. Raycast задаёт миру вопрос: «какой физический объект первым встретится на этом отрезке?»

В проекте при выстреле передаются:

Аргумент	Назначение
`origin`	мировая позиция дула
`direction`	направление выстрела
`world`	Bullet physics world
`range`	конечная дальность
`shooterEntity`	сущность, которую нельзя принять за цель
`shooterObject`	физическое тело стрелка для исключения
`rayHit`	структура результата

Не используй заранее выбранного `targetEntity` как доказательство попадания. Урон должен получить первый объект луча: стена, союзник или цель.

18 Угол поворота без скачка через 360 градусов

Обычное вычитание углов ломается рядом с границей `-pi` и `pi`. Кратчайшая разница вычисляется так:

$$\text{error} = \operatorname{atan2}(\sin(\text{target} - \text{current}), \cos(\text{target} - \text{current}))$$

```
const float yawError = std::atan2(
    std::sin(targetYaw - currentYaw),
    std::cos(targetYaw - currentYaw));
```

Результат всегда лежит в диапазоне от `-pi` до `pi`. Знак говорит, в какую сторону вращаться.

19 Время таймеры и плавное изменение

Таймер обратного отсчёта:

```
timer = (std::max)(0.0f, timer - deltaTime);
if (timer <= 0.0f)
{
    // Событие разрешено.
}
```

Линейная интерполяция:

$$\mathrm{lerp}(a, b, t) = a + (b - a)t$$

```
const float value = a + (b - a) * t;
```

Ограничивай `t` диапазоном 0..1, если не нужна экстраполяция. Для плавности, одинаковой при разном FPS, полезно экспоненциальное приближение:

$$\mathrm{alpha} = 1 - e\text{^}(-\mathrm{speed} \cdot \mathrm{deltaTime})$$

```
const float alpha = 1.0f - std::exp(-responseSpeed * deltaTime);
current += (target - current) * alpha;
```

20 Как математика связана с текущим NPC

Кодовая задача	Математика	Проверка в отладчике
найти ближайшего врага	квадрат расстояния	линия до выбранной цели
проверить поле зрения	нормализация и dot	конус или граничные лучи
подтвердить видимость	raycast	зелёный или красный луч
направить ствол	матрицы кости и мира	точка muzzle
запретить выстрел из стены	короткий ray chest to muzzle	луч упирается в укрытие
проверить линию огня	конечный raycast	первый пересечённый entity
оценить укрытие	несколько лучей на высотах тела	трассеры к кандидату
повернуться к угрозе	atan2 и кратчайшая разница	yaw error

21 Практика по математике

Задание 1 направление

NPC находится в `(3, 0, 2)`, цель в `(11, 0, -4)`. Вручную вычисли направление, длину и единичный вектор. Затем сравни с DirectXMath.

Задание 2 поле зрения

Направление NPC `(0, 0, 1)`. Проверь цели в направлениях `(0, 0, 1)`, `(1, 0, 1)` и `(1, 0, 0)` для FOV 90 градусов. Сначала предскажи результат.

Задание 3 матрицы

Вручную сложи и умножь матрицы:

$$A = [2,1;\ 0,3], B = [4,-1;\ 2,5]$$

Для умножения выпиши все четыре ячейки отдельно.

Задание 4 порядок трансформаций

Возьми точку `(1, 0, 0)`. Сравни два случая: сначала перенос `(5, 0, 0)`, потом поворот на 90 градусов; и сначала поворот, потом перенос. Нарисуй результат сверху.

Задание 5 собственный луч

Сделай маленькую функцию, которая принимает начало и конечную точку, возвращает нормализованное направление и длину. Она обязана отклонять почти нулевой отрезок.

Задание 6 визуальная проверка

Нарисуй в отладке три разных цвета:

- жёлтый от глаз NPC к кандидату;
- красный от груди к дулу;
- зелёный от дула до первого попадания.

По одному кадру должно быть понятно, какой именно тест запретил или разрешил выстрел.

22 Ответы для самопроверки

Для задания 1 направление равно `(8, 0, -6)`, длина равна `10`, нормализованный вектор равен `(0.8, 0, -0.6)`.

Для задания 2 первая цель точно внутри. Вторая лежит на границе 45 градусов и должна считаться внутри с небольшим допуском. Третья находится под 90 градусов к forward и снаружи.

Для задания 3:

$$A + B = [6,0;\ 2,8]$$

$$AB = [10,3;\ 6,15]$$

23 Контрольный список

Перед следующей частью ты должен уметь без подсказки:

- объяснить разницу между точкой и вектором;
- получить направление как `end - start`;
- безопасно нормализовать вектор;
- сравнить расстояния без квадратного корня;
- объяснить dot через угол;
- вручную сложить две матрицы;
- вручную вычислить каждую ячейку произведения матриц;
- объяснить, почему порядок матриц меняет результат;
- построить конечный луч от дула к точке;
- объяснить разницу между математическим лучом и физическим raycast;
- найти кратчайшую ошибку угла;
- обновить таймер независимо от FPS.

Построчный разбор систем DX Framework

Эта часть разбирает текущий снимок проекта, а не абстрактный учебный движок. Номера строк относятся к версии кода на момент сборки книги. После правок номера могут сместиться, поэтому главным ориентиром остаются имя файла, имя метода и сама строка.

1 Как читать систему до изменения кода

Читай каждый `Update` в пять проходов:

1. Какие сущности система перебирает.
2. Какие компоненты только читает, а какие изменяет.
3. Какие guard clauses досрочно прекращают обработку.
4. Где принимается решение и где оно исполняется.
5. Какие значения надо показать в инспекторе, чтобы увидеть причину решения.

Не пытайся сразу удержать весь файл в голове. Сначала проследи один конкретный путь: например, «NPC увидел цель, дошёл до края укрытия и сделал один выстрел».

2 ECSRegistry и перебор двух компонентов

Файл `src/ECS/ECSRegistry.h`, метод `ForEach<T1, T2>`.

```
template<typename T1, typename T2, typename Fn>
void ForEach(Fn&& fn)
{
    auto* pool1 = FindPool<T1>();
    auto* pool2 = FindPool<T2>();
    if (!pool1 || !pool2) return;
    for (auto& [entity, comp1] : *pool1)
    {
        if (pool2->Has(entity))
        {
            auto& comp2 = *pool2->Get(entity);
            fn(entity, comp1, comp2);
        }
    }
}
```

Строка	Что происходит	Зачем это здесь	Что сломается при ошибке
`template<typename T1, typename T2, typename Fn>`	Метод получает два типа компонентов и тип функции обратного вызова	Один код работает для любых пар компонентов	Жёстко заданные типы породят отдельный цикл на каждую систему
`void ForEach(Fn&& fn)`	Принимается callable без лишнего копирования	Лямбда может захватывать данные системы	Передача тяжёлой функции по значению создаст копии
`pool1 = FindPool<T1>()`	Ищется существующее хранилище первого типа	Перебирать можно только созданный pool	Создание pool при чтении скроет отсутствие компонентов
`pool2 = FindPool<T2>()`	Ищется хранилище второго типа	Оно будет проверяться для каждой сущности	Без него нельзя получить `comp2`
`if (!pool1 || !pool2) return`	Вся операция прекращается, если одного типа вообще нет в мире	Нет ни одного совпадения	Разыменование `nullptr` приведёт к падению

Строка	Что происходит	Зачем это здесь	Что сломается при ошибке
`for (... : *pool1)`	Перебирается только первый pool	Стоимость зависит от размера первого pool	Если первым выбран очень массовый компонент, цикл будет дороже
`pool2->Has(entity)`	Проверяется наличие второго компонента у той же сущности	Формируется пересечение двух множеств	Без проверки `Get` может вернуть `nullptr`
`comp2 = *pool2->Get(entity)`	Получается ссылка на второй компонент	Лямбда может читать и изменять его	Копия не сохранит изменения в ECS
`fn(entity, comp1, comp2)`	Управление передаётся системе	Registry не знает игровой логики	Неверный порядок аргументов даст путаницу типов

Важная деталь производительности

Этот `ForEach` всегда перебирает `pool1`. Поэтому полезно первым указывать более редкий компонент. Если `TransformComponent` есть у 10 000 сущностей, а `NPCCombatComponent` у 40, вызов `ForEach<NPCCombatComponent, TransformComponent>` выгоднее обратного порядка.

Мини задание

Найди вызов `ForEach<FactionComponent, TransformComponent>` в выборе цели. Ответь: сколько сущностей попадёт во внешний цикл и сколько проверок будет сделано, если Faction есть у 20 сущностей, а Transform у 5000.

3 TransformComponent и мировая матрица

Файл `src/ECS/Components/Components.h`, метод `GetWorldMatrix`.

```
XMMATRIX GetWorldMatrix() const
{
    XMMATRIX scaleMatrix = XMMatrixScaling(scale.x, scale.y, scale.z);
    XMMATRIX rotX = XMMatrixRotationX(rotation.x);
    XMMATRIX rotY = XMMatrixRotationY(rotation.y);
    XMMATRIX rotZ = XMMatrixRotationZ(rotation.z);
    XMMATRIX rotationMatrix = rotX * rotY * rotZ;
    XMMATRIX translationMatrix = XMMatrixTranslation(position.x, position.y, position.z);
    return scaleMatrix * rotationMatrix * translationMatrix;
}
```

Строка	Что происходит	Откуда берутся данные	Проверка
`GetWorldMatrix() const`	Строится матрица из текущего Transform	`position`, `rotation`, `scale` принадлежат компоненту	Метод не должен менять компонент
`XMMatrixScaling`	Создаётся масштаб по трём осям	`scale` по умолчанию `(1,1,1)`	Нулевой масштаб мешает обратной матрице
`XMMatrixRotationX`	Поворот вокруг X	`rotation.x` должен быть в радианах	Градусы дадут неверный огромный угол
`XMMatrixRotationY`	Поворот вокруг вертикальной оси	Для NPC это обычно yaw	Проверь знак и направление модели
`XMMatrixRotationZ`	Поворот вокруг Z	Roll или наклон	Неверная ось наклонит не туда
`rotX * rotY * rotZ`	Три поворота соединяются	Порядок является соглашением проекта	Перестановка изменит ориентацию
`XMMatrixTranslation`	Создаётся перенос в мировую позицию	`position` должна быть мировой	Локальная позиция родителя потребует иерархии

Строка	Что происходит	Откуда берутся данные	Проверка
`scale * rotation * translation`	Возвращается полный локально-мировой конвейер	DirectXMath использует выбранную проектом конвенцию	Другой порядок заставит объект вращаться вокруг иной точки

Как проверить без веры в формулу

Поставь объект в `(5, 0, 0)`, масштаб `(2, 1, 1)`, поворот Y на 90 градусов. Возьми локальную точку `(1, 0, 0)` и выведи её мировую позицию после `XMVector3TransformCoord`. Затем по одному отключай масштаб, поворот и перенос. Так видно вклад каждой матрицы.

4 NpcTacticalDecisionSystem и выбор стиля

Файл `NpcTacticalDecisionSystem.h`, метод `UpdateStyle`.

```
const float dx = threatPosition.x - npcPosition.x;
const float dz = threatPosition.z - npcPosition.z;
const float threatDistance = std::sqrt(dx * dx + dz * dz);

if (tactical.hasThreatDistanceSample && deltaTime > 0.0001f)
{
    tactical.enemyClosingSpeed =
        (tactical.lastThreatDistance - threatDistance) / deltaTime;
}
else
{
    tactical.enemyClosingSpeed = 0.0f;
    tactical.hasThreatDistanceSample = true;
}
tactical.lastThreatDistance = threatDistance;
```

Строка	Состояние до строки	Результат строки	Почему важно
`dx = threat.x - npc.x`	Есть две мировые позиции	Горизонтальная разница по X	Знак показывает сторону, квадрат убирает знак для длины
`dz = threat.z - npc.z`	То же по Z	Горизонтальная разница по Z	Y намеренно исключён из тактики дистанции
`sqrt(dx*dx + dz*dz)`	Есть катеты	Получено расстояние на плоскости	Для вывода скорости нужна настоящая длина, не квадрат
`hasThreatDistanceSample`	Первый кадр ещё не имеет истории	Определяется, можно ли сравнивать измерения	Без истории скорость сближения будет мусором
`deltaTime > 0.0001f`	Кадр мог быть паузой или нулевым шагом	Защита деления	Деление на ноль создаст бесконечность
`last - current`	Известны две дистанции	Положительный результат означает сближение	Обратный порядок поменяет смысл знака
`/ deltaTime`	Есть изменение за кадр	Получаются метры в секунду	Без деления значение зависит от FPS
`enemyClosingSpeed = 0`	Нет достоверной пары измерений	Безопасное начальное значение	NPC не принимает случайное решение в первый кадр
`hasThreatDistanceSample = true`	Первый образец принят	Следующий кадр сможет вычислить скорость	Флаг должен ставиться только после реального измерения
`lastThreatDistance = threatDistance`	Старое значение больше не нужно	Сохранена база для следующего кадра	Если записать раньше расчёта, скорость всегда будет нулём

Далее система уважает ручной выбор:

```
if (tactical.selectedStyle != NpcTacticalStyle::Auto)
{
    tactical.activeStyle = tactical.selectedStyle;
    tactical.tacticCommitTimer = 0.0f;
    tactical.tacticReason = "Стиль принудительно выбран в инспекторе";
    return;
}
```

Строка	Смысл
`selectedStyle != Auto`	Дизайнер хочет детерминированный тест, а не авто выбор
`activeStyle = selectedStyle`	Исполняемый стиль синхронизируется с настройкой
`commitTimer = 0`	Для ручного режима не нужна фиксация авто решения
`tacticReason = ...`	Инспектор объясняет источник решения
`return`	Ни одно авто правило ниже не имеет права перезаписать стиль

Затем вычисляются понятные булевы факты, а уже из них выбирается стиль. Это лучше одного гигантского `if`: каждый факт можно показать в отладчике и протестировать отдельно.

```
const bool enemyIsDangerouslyClose =
    threatDistance < (std::max)(2.5f, preferredRange * 0.35f);
const bool enemyIsCloseForCautiousNpc =
    threatDistance < preferredRange * 0.75f && aggression < 0.65f;
const bool enemyIsPushing = tactical.enemyClosingSpeed > 0.75f;
const bool cautiousUnderFire =
    tactical.underFireTime > 0.0f && aggression < 0.65f;
```

Порядок последующих `if` задаёт приоритет. Опасная близость должна проверяться раньше высокой агрессии, иначе агрессивный NPC продолжит пушить даже в неудобной почти рукопашной дистанции.

5 NpcCoverQuery и доказательство защиты

Файл `NpcCoverQuery.h`, метод `IsProtectedFromTarget`.

```
targetPosition.y += 1.0f;
coverPosition.y += (std::max)(0.2f, protectionHeight);

const XMFLOAT3 direction{
    coverPosition.x - targetPosition.x,
    coverPosition.y - targetPosition.y,
    coverPosition.z - targetPosition.z
};
const float distance = XMVectorGetX(
    XMVector3Length(XMLoadFloat3(&direction)));

if (distance <= 0.01f)
    return false;
```

Строка	Геометрический смысл	Почему не иначе
`target.y += 1.0f`	Источник поднимается примерно на уровень оружия	Луч от пола ошибочно признает низкий бордюр полной защитой
`cover.y += protectionHeight`	Проверяется конкретная высота защищаемого тела	Одна высота не доказывает защиту головы и груди одновременно
`direction = cover - target`	Луч направлен от угрозы к кандидату	Обратный вектор полетит от укрытия прочь от врага
`Length(direction)`	Получается конечная дальность луча	Бесконечный ray может подобрать геометрию за кандидатом

Строка	Геометрический смысл	Почему не иначе
`distance <= 0.01f`	Отклоняется вырожденная проверка	Нормализация и raycast нулевого отрезка ненадёжны

Продолжение:

```
    const auto* targetController =
        registry.GetComponent<CharacterController>(targetEntity);
    const btCollisionObject* targetBody =
        targetController ? targetController->ghostObj : nullptr;

    PhysicsRaycastHit hit{};
    PhysicsRaycast::CastRay(
        targetPosition,
        direction,
        world,
        distance,
        targetEntity,
        targetBody,
        hit);

    return hit.collisionObject &&
        hit.collisionObject->isStaticOrKinematicObject();
```

Строка	Что происходит	Возможная ошибка
`GetComponent<CharacterController>`	Ищется физическое представление цели	Не все сущности обязаны иметь controller
тернарный оператор	Безопасно выбирается `ghostObj` или `nullptr`	Разыменование отсутствующего controller уронит игру
`PhysicsRaycastHit hit{}`	Результат обнуляется перед вызовом	Старое попадание не должно пережить новый луч
`CastRay`	Проверяется только отрезок до позиции укрытия	Большая дальность даст ложную стену за точкой
`targetEntity`, `targetBody`	Цель исключается из собственного луча	Луч иначе столкнётся со стрелком в самом начале
`hit.collisionObject`	Должно существовать препятствие	Свободная линия означает отсутствие защиты
`isStaticOrKinematicObject()`	Защитой признаётся стабильная геометрия	Живой NPC не должен считаться стеной

Ограничение одной высоты

Этот метод отвечает только на вопрос про одну выбранную высоту. Для стойки нужны минимум лучи к груди и голове. Для приседа можно проверять более низкие точки. Поэтому будущая оценка укрытия должна возвращать не просто `bool`, а профиль: `standingProtected`, `crouchProtected`, `leftPeekValid`, `rightPeekValid`.

6 StateSystem и исполнение движения в укрытие

Файл `StateSystem.h`, ветка `NpcState::TakingCover`.

```
auto* tactical = registry.GetComponent<NpcTacticalComponent>(npc);
auto* combat = registry.GetComponent<NPCCombatComponent>(npc);
if (!tactical || !combat || !combat->hasLastKnownPosition)
{
    brain.state = NpcState::Chasing;
    break;
}
```

Строка	Кто владеет данными	Решение
получение `tactical`	Тактический компонент хранит цель движения и фазу укрытия	Без него состояние не может исполниться
получение `combat`	Боевой компонент хранит контакт	Укрытие оценивается относительно угрозы
guard clause	Проверяет минимальный контракт состояния	Нельзя продолжать с неполными данными
`state = Chasing`	Выбирается восстановительное состояние	Сейчас это политика; защитнику позже может подойти `Searching` или удержание позиции
`break`	Завершается только текущая ветка `switch`	`return` мог бы прекратить обработку всей сущности или системы в зависимости от контекста

Выбор режима перемещения:

```
NPCMovementComponent tacticalMovement = npcMovement;
if (tactical->movingToPeek)
{
    tacticalMovement.speed = (std::max)(
        tacticalMovement.speed,
        tactical->peekMoveSpeed);
    tacticalMovement.allowRunning = false;
}
else
{
    tacticalMovement.allowRunning = npcMovement.allowRunning;
    tacticalMovement.runWhenFartherThan = 0.0f;
}
```

Строка	Смысл	Почему создана копия
`tacticalMovement = npcMovement`	Берутся обычные настройки NPC	Исходный компонент не меняется навсегда
`movingToPeek`	NPC идёт к точке выглядывания, а не к безопасной точке	Эти движения должны выглядеть по-разному
`max(speed, peekMoveSpeed)`	Выглядывание не станет медленнее обычной ходьбы	Резкая анимация требует короткого перемещения
`allowRunning = false`	Выглядывание не переключает locomotion в Run	Позже его заменит отдельный lean или peek clip
ветка `else`	NPC реально бежит к укрытию	Под огнём медленная прогулка неверна
`runWhenFartherThan = 0`	Бег разрешён на любой ненулевой дистанции	Обычный порог дальности здесь слишком поздний

Навигация и прибытие:

```
const NavigationResult result = NavigateTo(
    registry, npc, brain, transform, movement,
    tacticalMovement, tactical->moveTarget,
    room, npcMovement.arrivalDistance,
    NpcState::TakingCover, deltaTime);

if (result == NavigationResult::Arrived)
{
    StopMovement(movement);
    if (tactical->movingToPeek)
    {
        tactical->exposed = true;
        tactical->crouched = false;
        tactical->phaseTimer = (std::max)(
            combat->rangedAimDelay + 1.2f,
            tactical->exposeDuration);
        brain.state = NpcState::Attacking;
    }
    else
    {
        tactical->exposed = false;
        tactical->crouched = tactical->coverUsesCrouch;
        tactical->phaseTimer = (std::max)(0.1f, tactical->hideDuration);
        brain.state = NpcState::InCover;
    }
}
```

Строка	Изменение состояния	Инвариант после строки
`NavigateTo`	Исполнитель пробует приблизиться к `moveTarget`	Решение о точке уже принято другой системой
`Arrived`	Путь закончен	Нельзя оставлять старую скорость
`StopMovement`	Скорость и флаг движения сбрасываются	Анимация не продолжает бег на месте
`movingToPeek`	Различаются край укрытия и защищённая позиция	Одна координата не должна означать обе фазы
`exposed = true`	Тактика знает, что NPC доступен для огня	Получение урона в этой фазе может увеличить `failedPeekCount`
`crouched = false`	Для текущего peek используется стойка	Будущий профиль укрытия сможет выбрать crouch-peek
`phaseTimer = max(...)`	Окно включает прицеливание и короткую очередь	Слишком короткий таймер вернёт NPC до выстрела
`state = Attacking`	Управление передаётся боевой ветке	Сам `TakingCover` не стреляет
защищённая ветка	NPC скрыт и ждёт	`phaseTimer` становится временем ожидания до peek
`state = InCover`	Управление передаётся фазе укрытия	Следующий переход должен быть явным

Если `NavigateTo` возвращает `Blocked`, резервирование сбрасывается, таймер решения обнуляется и поиск должен получить шанс выбрать другую точку. Это пример восстановления после ошибки исполнения, а не нового тактического решения.

7 NpcRangedCombatSystem и точка дула

Файл `src/Gameplay/Combat/NpcRangedCombatSystem.h`.

```
XMMATRIX hand;
if (!mesh->model->TryGetBoneModelTransform(
        *animation,
        weapon->rightHandBone,
        hand))
{
    combat.rangedStatus = "Missing right-hand bone";
    return;
}
```

Строка	Что проверяется	Почему выстрел запрещается
`XMMATRIX hand`	Подготавливается место для матрицы кости	Дуло привязано к текущей анимационной позе
`TryGetBoneModelTransform`	Модель ищет кость, заданную оружием	Имена костей отличаются между моделями
`!result`	Кость не найдена	Вычислять мировое дуло не из чего
`rangedStatus`	Причина сохраняется для инспектора	Иначе NPC молча перестанет стрелять
`return`	Текущая попытка выстрела прекращается	Подстановка нулевой матрицы даст луч из неверного места

Мировая точка дула:

```
XMStoreFloat3(
    &origin,
    XMVector3TransformCoord(
        XMLoadFloat3(&weapon->muzzlePosition),
        WeaponIKSystem::GripMatrix(weapon->worldGrip) *
            hand *
            transform->GetWorldMatrix()));
```

Читаем изнутри наружу:

1. `muzzlePosition` — точка в локальном пространстве оружия.
2. `GripMatrix` учитывает хват и ориентацию оружия.
3. `hand` переносит оружие в текущую позу кости.
4. `GetWorldMatrix` переносит персонажа в мир уровня.
5. `TransformCoord` применяет полный конвейер к точке.
6. `XMStoreFloat3` сохраняет SIMD-вектор в обычный `XMFLOAT3`.

Направление строится только к точке, подтверждённой личным зрением:

```
XMFLOAT3 aimPoint = combat.visibleAimPoint;
XMFLOAT3 direction{
    aimPoint.x - origin.x,
    aimPoint.y - origin.y,
    aimPoint.z - origin.z
};
```

`targetEntity` может оставаться активным после ухода игрока за стену. Поэтому он не является разрешением стрелять. `visibleAimPoint` должен обновляться только при текущем подтверждённом контакте.

8 Проверка chest to muzzle

```
XMFLOAT3 chest = transform->position;
chest.y += 1.45f;

const XMFLOAT3 toMuzzle{
    origin.x - chest.x,
    origin.y - chest.y,
    origin.z - chest.z
};

const float muzzleDistance = XMVectorGetX(
    XMVector3Length(XMLoadFloat3(&toMuzzle)));
```

Строка	Геометрия	Ошибка, которую ловит
`chest = position`	Берётся мировая база персонажа	Нужна стабильная точка внутри тела
`chest.y += 1.45`	Точка поднимается к груди	Луч от ног пересёк бы пол или низ стены
`origin - chest`	Направление идёт к дулу	Проверяем, не высунулся ли ствол сквозь геометрию
`Length`	Получается длина короткого отрезка	Raycast не должен идти дальше дула

После raycast любое препятствие на этом отрезке ставит статус `Muzzle blocked by cover` и запрещает выстрел. Эта проверка не заменяет линию огня: она отвечает только за участок внутри позы NPC и укрытия.

9 Проверка линии огня и союзника

```
PhysicsRaycastHit fireLane{};
PhysicsRaycast::CastRay(
    origin,
    direction,
    world,
    distance,
    npc,
    ownBody,
    fireLane);
```

Аргумент	Точное назначение
`origin`	мировое дуло
`direction`	ненормализованный вектор к видимой точке; обёртка raycast знает длину отдельно
`world`	физический мир
`distance`	конец проверки находится на цели, а не за ней
`npc`	сущность стрелка исключается
`ownBody`	физический ghost стрелка исключается
`fireLane`	первое попадание возвращается сюда

```
const auto* blockingNpc =
    registry.GetComponent<NPCComponent>(fireLane.entity);
const auto* allyFaction =
    registry.GetComponent<FactionComponent>(fireLane.entity);

const bool friendlyInFireLane =
    blockingNpc &&
    blockingNpc->health > 0.0f &&
    allyFaction &&
    GetFactionRelation(
        observerFaction->faction,
        allyFaction->faction) == FactionRelation::Friendly;
```

Булево выражение читается слева направо благодаря short circuit:

1. Первый объект луча должен быть NPC.
2. Он должен быть жив.
3. У него должна быть фракция.
4. Отношение фракций должно быть Friendly.

Только после успешной проверки указателя читаются его поля. Переставлять `blockingNpc->health` перед `blockingNpc` нельзя.

10 WeaponSystem и одна попытка выстрела

Файл `src/Gameplay/Combat/WeaponSystem.h`, метод `TryFire`.

```
if (!world || !registry.IsAlive(shooterEntity) ||
    !registry.HasComponent<ActiveComponent>(shooterEntity) ||
    !WeaponFiringRules::ValidRay(origin, direction))
    return false;
```

Проверка	Что гарантирует следующему коду
`world`	физический raycast можно вызвать
`IsAlive`	entity не уничтожен
`ActiveComponent`	выключенная сущность не стреляет
`ValidRay`	координаты конечны и направление ненулевое

Поиск оружия и перезарядка:

```
const Entity weaponEntity =
    GetEquippedWeaponEntity(registry, shooterEntity);
auto* weapon = registry.GetComponent<WeaponComponent>(weaponEntity);
if (!weapon)
    return false;

if (weapon->ammoInMagazine <= 0)
{
    WeaponFiringRules::StartReload(*weapon);
    return false;
}

if (!WeaponFiringRules::CanFire(*weapon))
    return false;
```

Строка	Состояние оружия	Результат
`GetEquippedWeaponEntity`	Стрелок может иметь инвентарь и несколько предметов	Находится именно экипированное оружие
`GetComponent<WeaponComponent>`	Entity мог быть неверным или предмет не оружие	Получается механическое состояние

Строка	Состояние оружия	Результат
`ammo <= 0`	Магазин пуст	Запускается таймер перезарядки
`return false`	В этот кадр пуля не создаётся	Перезарядка не считается выстрелом
`CanFire`	Проверяет кулдаун, reload и другие правила	Все стрелки используют один контракт

Разброс и фиксация выстрела:

```
XMStoreFloat3(&normalized,
    XMVector3Normalize(XMLoadFloat3(&direction)));

const auto* movement =
    registry.GetComponent<CharacterMovement>(shooterEntity);
const float motionScale = WeaponFiringRules::MotionMultiplier(
    *weapon,
    movement && movement->isMoving);

const float radial = sample(random);
const float angular = sample(random);
const XMFLOAT3 muzzleDirection = normalized;
normalized = WeaponFiringRules::ConeDirection(
    normalized,
    weapon->spreadDegrees * motionScale +
        weapon->currentRecoilDegrees,
    radial,
    angular);

WeaponFiringRules::CommitShot(*weapon, motionScale);
```

Строка	Почему расположена именно здесь
нормализация	Дальность и конус ожидают единичное направление
чтение движения	Разброс зависит от состояния стрелка
`movement && ...`	Отсутствующий компонент безопасно трактуется как неподвижность
две случайные выборки	Одна задаёт радиус, другая угол внутри конуса
сохранение `muzzleDirection`	Muzzle flash должен смотреть по стволу, а не по случайному отклонению пули
`ConeDirection`	Получается фактический луч этого выстрела
`CommitShot`	Списывается патрон, ставится кулдаун и отдача

Сначала фиксируется выстрел, затем делается raycast. Поэтому промах всё равно тратит патрон и создаёт кулдаун — это правильная механика.

Попадание превращается в событие:

```
PhysicsRaycastHit rayHit{};
const bool hit = PhysicsRaycast::CastRay(
    origin,
    normalized,
    world,
    weapon->range,
    shooterEntity,
    shooterObject,
    rayHit);

if (!hit)
    return true;

DamageEvent damageEvent{};
damageEvent.source = weaponEntity;
damageEvent.target = rayHit.entity;
damageEvent.damageAmount = weapon->damage;
damageSystem.AddDamageEvent(damageEvent);
return true;
```

`true` означает «выстрел состоялся», а не «цель получила урон». Это различие нужно для анимации, звука, расхода патронов и тактического поведения.

11 Один кадр боя от восприятия до урона

Полезно читать не отдельные классы, а движение данных:

1. Система восприятия подтверждает личную видимость и обновляет `visibleAimPoint`.
2. Боевая система хранит `targetEntity`, но не считает его вечной видимостью.
3. Тактическое решение выбирает стиль и запрос укрытия.
4. Cover query оценивает кандидатов лучами относительно подтверждённой угрозы.
5. `StateSystem` исполняет `moveTarget` через навигацию.
6. В состоянии атаки `NpcRangedCombatSystem` вычисляет мировое дуло.
7. Проверяются поворот, chest-to-muzzle и дружественная линия огня.
8. `WeaponSystem::TryFire` проверяет оружие, создаёт разброс и физический луч.
9. Первое попадание превращается в `DamageEvent`.
10. `DamageSystem` применяет урон и записывает факт угрозы для следующего тактического решения.

Если NPC не стреляет, проверяй цепочку сверху вниз. Не начинай сразу с `TryFire`: до него мог не дойти вызов.

12 Метод локализации бага

Для каждого перехода храни три строки отладки:

- `decision`: чего NPC хотел;
- `reason`: почему он этого захотел;
- `execution`: что исполнитель реально сделал.

Пример:

```
decision: перейти к укрытию 41
reason: два попадания во время выглядывания
execution: путь не найден, cover 41 отклонено на 4 секунды
```

Это различает ошибку решения и ошибку исполнения. Если решение правильное, но NPC стоит, смотри NavMesh и состояние движения. Если путь есть, но выбрана плохая сторона стены, смотри геометрию cover query и score.

13 Практика по чтению кода

Задание 1 ECS

Нарисуй два множества сущностей: с `NPCCombatComponent` и с `TransformComponent`. Руками отметь пересечение, которое получит лямбда.

Задание 2 матрица дула

В отладке покажи четыре точки разными цветами: локальное дуло после grip, после hand, после world и окончательный `origin`. Если одна стадия прыгает, ты сразу узнаешь виноватую матрицу.

Задание 3 укрытие

Для одного кандидата нарисуй лучи к ногам, груди и голове. В журнале напиши, какой луч встретил статическую геометрию первым.

Задание 4 выстрел

Поставь союзника между двумя врагами. Запиши по кадрам `fireLane.entity`, отношение фракций и `rangedStatus`. Затем убери союзника и сравни цепочку.

Задание 5 объяснение

Не глядя в книгу объясни вслух, почему активный `targetEntity` не даёт права стрелять. Правильный ответ должен упомянуть текущую личную видимость, укрытие и первое физическое попадание луча.

14 Контрольный список

После разбора ты должен уметь:

- отличить выбор сущностей ECS от игровой логики лямбды;
- назвать пространство каждой точки в расчёте дула;
- объяснить, почему `lastKnownPosition` не является всезнанием;
- разделить решение об укрытии, поиск геометрии и исполнение движения;
- прочитать guard clause как контракт следующего участка;
- отличить «выстрел состоялся» от «луч попал»;
- найти, какая система владеет изменяемым полем;
- построить цепочку данных одного кадра;
- добавить отладку причины, а не только результата.

Справочник полезных алгоритмов

Этот раздел нужен не для заучивания. Перед новой механикой найди похожую задачу, выпиши входы и выходы, затем реализуй минимальный вариант и обязательно добавь визуальную проверку.

1 Безопасная нормализация

Задача: получить единичное направление без `NaN`.

Вход: произвольный вектор `v`.

Выход: единичный вектор и признак успеха.

```
bool TryNormalize(const XMFLOAT3& value, XMFLOAT3& result)
{
    const XMVECTOR vector = XMLoadFloat3(&value);
    const float lengthSquared = XMVectorGetX(XMVector3LengthSq(vector));
    if (lengthSquared <= 0.000001f)
        return false;

    XMStoreFloat3(&result, XMVector3Normalize(vector));
    return true;
}
```

Сложность: O(1).

Ловушка: проверять длину уже после нормализации.

Применение: лучи зрения, движение, направление ствола, steering.

2 Квадрат расстояния

Задача: быстро сравнить кандидатов или радиус.

```
float DistanceSquared(const XMFLOAT3& a, const XMFLOAT3& b)
{
    const float dx = a.x - b.x;
    const float dy = a.y - b.y;
    const float dz = a.z - b.z;
    return dx * dx + dy * dy + dz * dz;
}
```

Сравнивай с `radius * radius`. Квадратный корень нужен только для настоящего расстояния.

Сложность: O(1) на кандидата, O(n) для полного перебора.

3 Поле зрения через dot

Вход: нормализованный `forward`, направление к цели, полный угол FOV.

```
const float halfFov = XMConvertToRadians(fovDegrees * 0.5f);
const float threshold = std::cos(halfFov);
const bool inside = Dot(forward, normalizedToTarget) >= threshold;
```

Ловушка: использовать полный угол вместо половинного.

Применение: дешёвый фильтр перед raycast. После dot всё равно проверяется препятствие.

4 Кратчайшая разница углов

```
float ShortestAngle(float current, float target)
{
    return std::atan2(
        std::sin(target - current),
        std::cos(target - current));
}
```

Выход: значение от `-pi` до `pi`.

Применение: поворот NPC, турели, оценка ошибки прицеливания.

5 Луч и отрезок

Луч бесконечен при `t >= 0`. Для оружия и зрения обычно нужен отрезок с максимальным расстоянием.

```
struct SegmentRay
{
    XMFLOAT3 origin{};
    XMFLOAT3 direction{}; // Единичный вектор.
    float maxDistance = 0.0f;
};
```

Создание из двух точек:

1. `delta = end - start`;
2. `distance = length(delta)`;
3. отклонить почти нулевую длину;
4. `direction = delta / distance`;
5. `maxDistance = distance`.

Ловушка: передать ненормализованное направление в функцию, которая считает `t` метрами.

6 Ray plane

Формула:

$$t = ((Q - O) \cdot N)/(D \cdot N)$$

Порядок: проверить параллельность, получить `t`, проверить диапазон, вычислить точку.

Сложность: O(1).

Применение: приказ на поверхность, маркер мыши, проекция тактической точки.

7 Ray sphere

Порядок: построить квадратное уравнение, проверить дискриминант, выбрать ближайший неотрицательный корень.

Сложность: O(1).

Применение: грубый broad phase, выбор персонажа, отладочные цели.

Ловушка: вернуть отрицательный корень, лежащий позади начала.

8 Ray AABB slab method

Порядок: для каждой оси найти интервал `t1..t2`, пересечь его с общим `tMin..tMax`, отклонить пустое пересечение.

Сложность: O(1), ровно три оси.

Применение: редактор, broad phase, грубая проверка видимости объекта.

Ловушка: делить на почти нулевую компоненту направления. Параллельная ось проверяется отдельно.

9 Lerp

$$\mathrm{lerp}(a, b, t) = a + (b - a)t$$

Применение: цвет, позиция, числовой параметр.

Ловушка: использовать один и тот же `t` каждый кадр как будто это постоянная скорость. `current = lerp(current, target, 0.1)` зависит от FPS.

10 Экспоненциальное приближение

```
const float alpha = 1.0f - std::exp(-speed * deltaTime);
current += (target - current) * alpha;
```

Применение: камера, плавный поворот, UI, шумные оценки AI.

Плюс: почти одинаковая реакция при разном FPS.

11 Fixed timestep

Физику удобно обновлять фиксированным шагом.

```
accumulator += frameDelta;
while (accumulator >= fixedDelta)
{
    SimulateFixedStep(fixedDelta);
    accumulator -= fixedDelta;
}
```

Ловушка: бесконечно догонять после долгой паузы. Ограничивай `frameDelta` и число шагов.

Применение: физика, детерминированные таймеры, воспроизводимые тесты.

12 Cooldown и one shot trigger

```
cooldown = (std::max)(0.0f, cooldown - deltaTime);
if (request && cooldown <= 0.0f)
{
    ExecuteAction();
    cooldown = cooldownDuration;
}
```

Применение: выстрел, смена укрытия, повторный звук, пересчёт пути.

Ловушка: ставить cooldown каждый кадр, пока запрос активен. Тогда он никогда не достигнет нуля.

13 Конечный автомат

Минимальная модель:

```
Idle -> Searching -> Chasing -> TakingCover -> InCover -> Attacking
```

Для каждого состояния зафиксируй:

- условия входа;
- действия во время состояния;
- условия выхода;
- обязательные компоненты;
- восстановительное состояние при ошибке.

Сложность: O(1) на сущность за кадр, если состояние исполняет ограниченную работу.

Ловушка: менять состояние в нескольких системах без единого владельца перехода.

14 Utility scoring

Каждое действие получает нормализованный score.

$$\text{score} = \text{w_1f_1} + \text{w_2f_2} + \ldots + \text{w_nf_n}$$

Пример выбора укрытия:

```
score = protection * 5
      + rangeFit * 2
      + peekQuality * 3
      - pathCost * 0.2
      - exposureDuringMove * 4
```

Порядок: сначала жёсткие фильтры, затем score. Нельзя компенсировать отсутствующую защиту красивой дистанцией.

Ловушка: факторы разных масштабов. Приводи их примерно к 0..1 или явно учитывай диапазоны весов.

15 Гистерезис

Гистерезис не даёт переключаться между почти равными решениями.

```
if (newScore > currentScore + switchMargin)
    SelectNewCandidate();
```

Можно добавить минимальное время фиксации решения.

Применение: стиль боя, выбор укрытия, оружия, цели.

Ловушка: лечить гистерезисом плохие входные данные. Сначала проверь score и геометрию.

16 A star

A star ищет путь по графу.

$$f(n) = g(n) + h(n)$$

`g` — уже пройденная стоимость, `h` — оценка оставшегося пути.

Структуры: open set, closed set, `cameFrom`, стоимость `g`.

Сложность: зависит от графа и очереди; с приоритетной очередью обычно близка к O(E log V).

Применение: NavMesh или сетка.

Ловушки:

- пересчитывать полный путь каждый кадр;
- считать точку достижимой только потому, что она близко геометрически;
- не инвалидировать путь при смене `moveTarget`;
- не различать `NoPath`, `Blocked`, `Arrived`.

17 Arrival steering

Обычный seek летит к цели на полной скорости и перелетает её. Arrival снижает скорость рядом с целью.

```
const float distance = Length(toTarget);
const float desiredSpeed = maxSpeed *
    (std::min)(1.0f, distance / slowRadius);
const Vec3 desiredVelocity = Normalize(toTarget) * desiredSpeed;
```

Применение: остановка у укрытия и точки ремонта.

Ловушка: `slowRadius` меньше `arrivalDistance` или равен нулю.

18 Separation

Для каждого близкого соседа создаётся отталкивание, обычно сильнее на малой дистанции.

```
separation += away / max(distanceSquared, epsilon)
```

Применение: NPC не слипаются на пути.

Ловушка: запускать полный O(n^2) перебор для тысяч сущностей. Используй spatial hash или соседей навигации.

19 Spatial hash

Мир делится на ячейки. Координата превращается в целочисленный ключ:

```
cellX = floor(position.x / cellSize);
cellZ = floor(position.z / cellSize);
```

Для поиска в радиусе проверяются соседние ячейки, а не все сущности мира.

Средняя сложность: близка к O(1 + k), где k — число локальных кандидатов, при удачном размере ячейки.

Применение: восприятие, separation, поиск ближайших объектов.

20 Случайное направление внутри конуса

Равномерный разброс по площади нельзя получать равномерным углом по двум осям: это искажает плотность.

Практическая схема:

1. две независимые выборки `u` и `v` от 0 до 1;
2. радиальная часть использует `sqrt(u)`;
3. угол равен `2*pi*v`;
4. строится локальное отклонение;
5. оно переносится в базис `right`, `up`, `forward`.

Применение: `WeaponFiringRules::ConeDirection`.

Ловушка: делать новую случайную выборку отдельно для дебаг-луча и урона. Один выстрел должен иметь одно фактическое направление.

21 Семплирование динамических укрытий

Если авторские точки не покрывают все варианты боя, кандидаты могут генерироваться вокруг NPC или препятствий.

Кольцо вокруг NPC

```
for (int i = 0; i < sampleCount; ++i)
{
    const float angle = twoPi * i / sampleCount;
    candidate.x = npc.x + std::cos(angle) * radius;
    candidate.z = npc.z + std::sin(angle) * radius;
}
```

Для каждого кандидата:

1. спроецировать на NavMesh;
2. проверить достижимость;
3. проверить защиту несколькими лучами;
4. найти левый и правый peek;
5. оценить расстояние до угрозы;
6. оценить открытость пути;
7. применить score и hysteresis.

Ловушка: считать точку укрытием только потому, что один низкий луч встретил стену.

22 Профиль укрытия вместо bool

Полезная структура результата:

```
struct CoverProfile
{
    bool standingProtected = false;
    bool crouchingProtected = false;
    bool leftPeekValid = false;
    bool rightPeekValid = false;
    float pathCost = 0.0f;
    float exposureCost = 0.0f;
    float score = -FLT_MAX;
};
```

Так решение не теряет информацию. Анимационный модуль позднее выберет стойку и сторону выглядывания, не повторяя геометрический поиск.

23 Reservoir sampling для случайного кандидата

Если кандидатов много и надо равновероятно выбрать один без массива:

```
int seen = 0;
Entity selected = INVALID_ENTITY;
for (Entity candidate : candidates)
{
    ++seen;
    if (RandomInt(1, seen) == 1)
        selected = candidate;
}
```

Применение: разнообразие патрульных точек или равных укрытий.

24 Ring buffer журнала решений

Храни последние N событий в кольцевом буфере, а не одну быстро меняющуюся строку.

```
timestamp | state before | decision | reason | state after
```

Сложность: O(1) на запись, фиксированная память.

Применение: NPC Logic Studio. Это решает проблему мелькающих статусов, которые невозможно прочесть даже в пошаговом режиме.

25 Приоритетная очередь событий угрозы

События можно оценивать по важности и свежести:

```
priority = damageWeight * damage
         + proximityWeight * proximity
         + recencyWeight * freshness
```

Тихое убийство может не создавать общую тревогу, но локальное событие попадания всё равно разворачивает выжившего NPC к близкой угрозе.

Ловушка: передавать точную текущую позицию врага через событие навсегда. Получатель должен подтвердить контакт собственным зрением.

26 Чек лист выбора алгоритма

Перед реализацией ответь:

1. Какие точные входы?
2. Какой точный выход?
3. В каком пространстве координат данные?
4. Нужен ли точный результат или дешёвый фильтр?
5. Сколько сущностей участвует?
6. Какова сложность на кадр?
7. Как алгоритм ведёт себя на нулевых и граничных значениях?
8. Как визуально доказать его результат?
9. Какая система владеет состоянием?
10. Что записать в журнал причины?

Учебный маршрут по DX Framework и NPC

Цель маршрута — не «прочитать побольше», а научиться объяснять код, предсказывать его поведение и самостоятельно добавлять механики без разрастания одной огромной системы.

1 Правило одного учебного блока

Каждый блок длится от 60 до 90 минут:

1. 20 минут теории.
2. 20 минут чтения текущего кода.
3. 30 минут маленькой самостоятельной реализации.
4. 10 минут теста и записи вывода.

После двух блоков делай перерыв. В конце дня сохраняй не только код, но и короткий отчёт:

```
Что хотел получить
Что предсказал до запуска
Что увидел
Какая строка объясняет результат
Что осталось непонятным
```

2 Интенсив на один свободный день

Это лучший старт перед четырёхнедельным маршрутом.

Блок 1 Векторы и луч

Теория: точка, вектор, `end - start`, длина, нормализация, конечная дальность.

Чтение: `NpcRangedCombatSystem.h`, участок от `visibleAimPoint` до вычисления `distance`.

Практика: вручную посчитать направление для двух точек, затем вывести значение DirectXMath.

Тест: нулевая длина, цель впереди, цель позади, цель выше.

Критерий готовности: можешь без подсказки объяснить каждый аргумент `CastRay`.

Блок 2 Матрица дула

Теория: матрица 4 на 4, сложение, умножение строка на столбец, порядок преобразований.

Чтение: `TransformComponent::GetWorldMatrix` и цепочка `Grip * hand * world`.

Практика: нарисовать промежуточные точки преобразования дула разными цветами.

Тест: повернуть NPC, сменить масштаб, проиграть анимацию, убедиться, что дуло следует за рукой.

Критерий готовности: можешь назвать пространство точки после каждого множителя.

Блок 3 Отладка одного выстрела

Теория: guard clause, первое попадание, дружественная линия огня.

Чтение: `NpcRangedCombatSystem` от chest-to-muzzle до `TryFire`.

Практика: поставить стену и союзника в разные участки луча. До запуска записать ожидаемый статус.

Тест: свободное дуло, заблокированное дуло, союзник в линии, враг за стеной.

Критерий готовности: по журналу видно точную первую причину запрета.

Блок 4 Маленькое самостоятельное изменение

Практика: добавить только новое поле отладки, не новую механику. Например, вывести `yawError`, `muzzleDistance`, `fireLane.entity`.

Проверка: значение сохраняется достаточно долго или попадает в ring buffer, поэтому его можно прочесть.

Что прислать на разбор: изменённый участок, один скрин инспектора, собственное объяснение из пяти предложений.

3 Неделя 1 Математика которую видно

День 1 Точки и векторы

Теория: сложение, вычитание, масштабирование.

Код: расчёты `dx`, `dy`, `dz` в боевых системах.

Задание: функция направления и длины между точками с защитой нулевого случая.

Выходной тест: пять наборов координат, включая одинаковые точки.

День 2 Dot и поле зрения

Теория: скалярное произведение, cosine, половинный FOV.

Код: текущая проверка видимости NPC.

Задание: нарисовать границы FOV и луч к кандидату.

Критерий: цель за стеной проходит FOV, но отклоняется raycast; ты можешь объяснить оба этапа.

День 3 Матрицы вручную

Теория: индексы, сложение ячеек, строка на столбец.

Задание: решить четыре матрицы 2 на 2 на бумаге, затем написать учебную функцию умножения без DirectXMath.

Критерий: все ячейки совпадают с ответом, порядок A B и B A даёт разные результаты.

День 4 Трансформации

Теория: scale, rotation, translation, world и inverse world.

Код: `GetWorldMatrix`.

Задание: преобразовать локальную контрольную точку и вернуть её обратной матрицей.

Критерий: возвращённая точка совпадает с исходной с малой погрешностью.

День 5 Лучи

Теория: параметрическое уравнение, ray против segment, ray-plane и ray-AABB.

Задание: отладочный луч из двух точек и ограниченная длина.

Критерий: препятствие за конечной точкой не считается попаданием.

День 6 Время

Теория: `deltaTime`, cooldown, fixed step, экспоненциальное сглаживание.

Задание: одинаковый таймер при 30, 60 и 144 FPS в тестовой симуляции.

Критерий: расхождение не превышает один шаг симуляции.

День 7 Проверка недели

Без книги объясни: нормализацию, dot, умножение матриц, порядок world transform, создание луча и таймер. Повтори только темы, которые не можешь объяснить на собственном примере.

4 Неделя 2 Архитектура текущего кода

День 8 ECS и владельцы данных

Код: `ECSRegistry::ForEach`, `GetComponent`, компоненты NPC.

Задание: таблица «поле → владелец → кто читает → кто пишет» для `targetEntity`, `lastKnownPosition`, `activeStyle`, `moveTarget`, `ammoInMagazine`.

Критерий: у каждого поля один понятный владелец записи.

День 9 Guard clauses

Код: `WeaponSystem::TryFire`.

Задание: для каждой ранней остановки записать, какой контракт она защищает.

Тест: искусственно вызвать каждую остановку и увидеть отдельную причину.

День 10 Состояния

Код: ветки `TakingCover`, `InCover`, `Attacking`.

Задание: нарисовать переходы и подписать условие каждого ребра.

Критерий: нет перехода «просто потому что так получилось».

День 11 Решение против исполнения

Код: `NpcTacticalDecisionSystem`, `NpcCoverQuery`, `StateSystem`.

Задание: подписать для каждой системы: вход, выход и запрещённые обязанности.

Критерий: query не меняет state, decision не строит путь, state не пересчитывает тактику.

День 12 Журнал решений

Задание: спроектировать ring buffer последних 64 решений с временем, состоянием до и после, причиной и entity кандидата.

Тест: пошаговый режим не теряет короткие промежуточные статусы.

День 13 Одна трасса кадра

Проследи один выстрел от восприятия до `DamageEvent`. Запиши функцию, поле и значение на каждом шаге.

День 14 Рефакторинг без изменения поведения

Выбери один длинный участок и выдели маленькую чистую функцию проверки. До и после сравни одинаковый сценарий и журнал.

Критерий: поведение не изменилось, причина стала читаться легче.

5 Неделя 3 Индивидуальный боевой NPC

День 15 Подтверждение контакта

Теория: perceived data против world truth.

Задание: проверить, что союзник передаёт только сообщение о контакте, а получатель сам подтверждает видимость.

Критерий: NPC не знает точную текущую позицию скрытого врага.

День 16 Выбор цели

Задание: разделить жёсткие фильтры и score цели: жив, враждебен, допустимая комната, расстояние, угроза.

Тест: игрок и NPC одновременно являются кандидатами.

День 17 Профиль укрытия

Задание: на бумаге спроектировать `CoverProfile` для стойки, приседа, левого и правого peek. Код писать только после проверки структуры.

Критерий: низкая стена не считается защитой стоящего NPC, но может защитить сидящего.

День 18 Score укрытия

Задание: жёстко отклонять незащищённые точки, остальные оценивать по защите, пути, дистанции и peek.

Тест: два NPC по одну сторону стены не считают её защитой друг от друга.

День 19 Фазы укрытия

Задание: записать таймлайн `move → hide → peek → aim → burst → return`.

Критерий: таймер экспозиции не истекает до завершения прицеливания.

День 20 Смена укрытия

Задание: смена только по понятным причинам: укрытие потеряло защиту, путь заблокирован, серия опасных peek, враг критически близко.

Критерий: NPC не бегает между укрытиями каждые две секунды без причины.

День 21 Дистанция и отступление

Задание: Defensive удерживает диапазон, Assault получает право сближаться, Balanced использует гистерезис.

Тест: при пуше одного NPC второй не повторяет симметричный пуш автоматически.

6 Неделя 4 Дружественные NPC и расширяемые модули

День 22 Память контакта

Спроектируй данные: кто сообщил, когда, где был подтверждён контакт, уверенность, время устаревания. Точная мировая позиция скрытого врага не читается напрямую.

День 23 Локальная реакция на выстрел

NPC слышит или получает попадание, поворачивается к близкому источнику, но не получает вечную цель. Оставь отдельное правило тишины для будущих тихих убийств.

День 24 Дружественная линия огня

Проверь перестроение союзника, пересечение траекторий и запрет огня. Добавь в журнал entity блокирующего союзника и длительность блокировки.

День 25 Простые роли без группового разума

Добавь роль как параметр индивидуальной тактики: Assault, Defensive, Support. Роль меняет веса и пороги, но не телепатически координирует NPC.

День 26 Контракт тактического модуля

Новый модуль должен возвращать предложение:

```
struct TacticalProposal
{
    TacticalAction action;
    float score;
    const char* reason;
};
```

Арбитр выбирает предложение, а исполнитель выполняет. Это позволяет добавлять Retreat, Suppress, Flank, ReloadInCover как пазлы.

День 27 Полигон

Создай сценарии:

- открытая площадка;
- высокая стена;
- низкая стена;
- двустороннее укрытие;
- узкий проход с союзником;
- две альтернативы пути;
- потеря контакта;
- противник быстро сближается.

Каждый сценарий должен иметь ожидаемый результат до запуска.

День 28 Вертикальный срез

Один NPC должен:

1. обнаружить врага;
2. подтвердить линию зрения;
3. выбрать роль и укрытие;
4. добежать с подходящей скоростью;
5. выбрать стойку и сторону peek;
6. проверить дуло и союзника;
7. сделать ограниченную очередь;
8. перезарядиться за укрытием;
9. сменить позицию только по объяснимой причине;
10. потерять контакт без всезнания.

7 Очерёдность будущих модулей

Добавляй по одному модулю и не смешивай их правила:

Порядок	Модуль	Вход	Предложение
1	Reload safety	патроны, укрытие, угроза	перезарядиться сейчас
2	Retreat	дистанция, здоровье, путь	увеличить дистанцию
3	Suppression	последняя подтверждённая зона, боезапас	подавлять область
4	Flank	профиль укрытия врага, маршруты	обойти сбоку
5	Grenade	группа целей, траектория, союзники	бросить гранату
6	Revive	раненый союзник, риск пути	помочь союзнику
7	Squad coordinator	предложения отдельных NPC	распределить роли

Групповая тактика идёт последней. Сначала одиночный NPC должен быть предсказуемым и объяснимым.

8 Формат мини задания для совместной проверки

Когда присылаешь решение, достаточно пяти частей:

1. Что хотел изменить.
2. Какое условие написал своими словами.
3. Сам изменённый участок кода.
4. Что ожидал до запуска.
5. Что реально произошло и какой статус был в журнале.

Так можно проверить твоё мышление, не отбирая у тебя самостоятельную работу готовым большим куском кода.

9 Критерии окончания маршрута

Маршрут закончен, когда ты можешь:

- вручную построить вектор, матрицу и луч;
- построчно объяснить выбранную систему;
- назвать владельца каждого изменяемого поля;
- отделить восприятие от истинного состояния мира;
- отделить решение от запроса геометрии и исполнения;
- добавить новый тактический модуль через общий контракт;
- доказать решение NPC журналом и трассерами;
- написать тестовый сценарий до запуска;
- найти баг по цепочке данных, а не случайными правками.